

Homework 2: Ping and Traceroute

CSCI-651 · Step-by-step plan · Due Fri Oct 9, 2026, 11:59 PM

Overview

Today is Sunday, Oct 4, so you have 5 days. Total effort is about **12-14 hours**, split into 1-3 hour chunks that fit around other assignments. Aim to be done by **Thursday night** and keep Friday as buffer.

Day	Work	Time
Sun Oct 4	Setup, start ping	~2 h
Mon Oct 5	Finish ping	~2 h
Tue Oct 6	Traceroute core	~2 h
Wed Oct 7	Finish traceroute, start docs	~2.5 h
Thu Oct 8	Docs, report, screenshots, zip	~2.5 h
Fri Oct 9	Buffer, submit early	0-1 h

Deliverables checklist

- **my_ping.py** and **my_traceroute.py** (plus any helper files, e.g. utils.py)
- **requirements.txt** (list Sphinx and anything else you install)
- **README.md** (how to run, example commands for every flag)
- **Report PDF** with screenshots of every flag for both programs
- **Code documentation PDF** (generated with Sphinx or similar)
- **revisions.txt** (output of `git log`)
- All in one zip named **firstname_lastname_hw2.zip** (at least 7 files)

Phase 0: Setup (~45 min, Sunday)

#	Task	Time
1	Create project folder, run <code>git init</code> , commit early and often (the git log is graded).	10 min
2	Create a virtualenv and <code>pip install sphinx flake8</code> .	10 min
3	Confirm you can open a raw socket. This needs root/admin: <code>sudo python3 my_ping.py . . .</code> on Linux/macOS, admin terminal on Windows (also allow inbound ICMP in the firewall). A Linux VM or WSL is easiest.	15 min
4	Skim the ping and traceroute man pages (linked in the assignment), focusing on output format.	10 min

Phase 1: Build my_ping.py (~3-3.5 h, Sun/Mon)

#	Task	Time
1	Argparse: destination, -c count, -i wait (default 1), -s packetsize (default 56), -t timeout.	20 min

#	Task	Time
2	Build the ICMP Echo Request: type 8, code 0, identifier, sequence number, payload padded to -s bytes, internet checksum.	45 min
3	Raw socket send/receive: skip the IP header, check type 0 (Echo Reply) and matching ID, read TTL from the IP header, compute RTT with perf_counter().	45 min
4	Real-ping output format, including the header line.	30 min
5	Ctrl+C handling and statistics: sent/received, % loss, min/avg/max/mdev.	30 min
6	Make -c and -t stop the program properly; handle timeouts and bad hostnames.	30 min

Core pieces (reference sketch)

```
def checksum(data: bytes) -> int:
    if len(data) % 2:
        data += b"\x00"
    total = sum((data[i] << 8) + data[i + 1] for i in range(0, len(data), 2))
    total = (total >> 16) + (total & 0xFFFF)
    total += total >> 16
    return ~total & 0xFFFF

# header: type(1) code(1) checksum(2) id(2) seq(2)
header = struct.pack("!BBHH", 8, 0, 0, ident, seq)
cs = checksum(header + payload)
packet = struct.pack("!BBHH", 8, 0, cs, ident, seq) + payload
```

Expected output (match the real ping):

```
PING example.com (93.184.216.34) 56(84) bytes of data.
64 bytes from 93.184.216.34: icmp_seq=1 ttl=57 time=12.3 ms

--- example.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
rtt min/avg/max/mdev = 11.9/12.3/12.8/0.4 ms
```

Test against 127.0.0.1, your router, and a public host (8.8.8.8 or google.com). Compare with the real ping. Commit after every step.

Phase 2: Build my_traceroute.py (~3.5-4 h, Tue/Wed)

#	Task	Time
1	Argparse: destination, -n, -q nqueries (default 3), -S.	15 min
2	Two sockets: a UDP socket to send probes (set TTL with setsockopt IP_TTL, destination port starting at 33434) and a raw ICMP socket to receive, with a 3-5 s timeout.	45 min
3	Hop loop: TTL 1 to 30, send -q probes per hop, time each one. ICMP type 11 = router replied (record its IP). ICMP type 3 code 3 = destination reached, stop. Timeout prints *.	60 min
4	Output format; reverse DNS with gethostbyaddr; with -n skip DNS and print only the IP. Header line.	45 min
5	-S summary: how many probes went unanswered for each hop.	30 min
6	Testing: several destinations, -q 1, -q 5, -n, -S; compare with real traceroute.	45 min

Expected output

```
traceroute to example.com (93.184.216.34), 30 hops max, 60 byte packets
 1 router.local (192.168.1.1)  1.2 ms  1.1 ms  1.0 ms
 2 10.0.0.1 (10.0.0.1)  5.4 ms  5.2 ms  5.9 ms
 3 * * *
 4 93.184.216.34 (93.184.216.34)  14.0 ms  13.8 ms  14.2 ms
```

Tip: the ICMP error contains the original IP header plus the first 8 bytes of your UDP packet. Use that to confirm the reply belongs to your probe (match the destination port).

Phase 3: Clean-up and documentation (~2.5 h, Wed/Thu)

#	Task	Time
1	PEP 8 and comments: run flake8 or pycodestyle, add docstrings to every function.	45 min
2	Sphinx PDF: sphinx-quickstart, enable sphinx.ext.autodoc, then make latexpdf (needs LaTeX). If LaTeX is a pain, build HTML and print to PDF.	45 min
3	README.md: how to run (with sudo) and examples for every flag.	20 min
4	requirements.txt (can be nearly empty if you only use the standard library, but list Sphinx).	5 min
5	Screenshots: my_ping with -c, -i, -s, -t; my_traceroute with -n, -q, -S.	30 min

Phase 4: Report and packaging (~1 h, Thu)

#	Task	Time
1	Report PDF: labeled screenshot per flag with one sentence of explanation each.	30 min
2	git log > revisions.txt	5 min
3	Zip as firstname_lastname_hw2.zip (check all 7+ files are inside).	15 min
4	Unzip in a different folder and run both programs once from scratch.	10 min

Sample commands to screenshot

```
sudo python3 my_ping.py -c 4 google.com
sudo python3 my_ping.py -c 5 -i 0.5 8.8.8.8
sudo python3 my_ping.py -c 3 -s 200 8.8.8.8
sudo python3 my_ping.py -t 5 8.8.8.8

sudo python3 my_traceroute.py google.com
sudo python3 my_traceroute.py -n google.com
sudo python3 my_traceroute.py -q 1 google.com
sudo python3 my_traceroute.py -S -q 5 google.com
```

Common pitfalls

- **Permission denied:** raw sockets need sudo / admin.
- **No replies on Windows:** allow inbound ICMP in the firewall.
- **Checksum errors:** compute the checksum with the checksum field set to 0, then repack.
- **Wrong replies matched:** always check the ICMP identifier (ping) or the embedded UDP port (traceroute).
- **Packet size off by 8:** -s is data bytes; add 8 for the ICMP header and 20 for the IP header ($56 + 8 = 64$, $+20 = 84$).
- **Traceroute stuck at * * *:** some routers drop probes; this is normal, keep going to the next TTL.
- **One giant commit:** make a commit after every step so revisions.txt shows real history.

Time-saving tips

- Write docstrings as you code so Sphinx is quick.
- Take screenshots right after testing each flag.
- Put the checksum and packet-building code in a shared utils.py.
- Compare against real ping and traceroute on the same target.
- Submit Thursday night or Friday morning, not at 11:50 PM.